

PAIMANA Prism: Team Project Documentation

Purpose: Presentation preparation based on the current repository implementation. This document describes what the code currently does, including its prototype and demo-data limitations. It does not describe planned functionality as if it were complete.

1. Project Introduction

Project name

PAIMANA Prism is a React and FastAPI decision-support dashboard for monitoring infrastructure project delivery risk.

Problem statement

Infrastructure monitoring information is difficult to use when it is spread across periodic reports, project fields, state summaries, and intervention notes. A decision-maker needs a concise view of which projects require attention, why a project is flagged, and what practical intervention may help.

In this repository, that problem is demonstrated through a dashboard that combines:

- Official PAIMANA state-summary data retrieved from the public PAIMANA portal.
- Clearly labelled synthetic project and project-month records for the prototype.
- Project risk scoring and plain-language reasons.
- A teammate intervention knowledge base used for recommendation matching.
- A what-if view for testing assumed delay and cost scenarios.

Objective

The implemented objective is to turn project-monitoring inputs into an authenticated dashboard where an administrator can:

1. Review portfolio and state-level signals.
2. Open a project and inspect its warning explanation.
3. View a deterministic intervention recommendation.
4. Add or edit a project record and obtain a risk score.
5. Explore a simple delay/cost scenario.
6. Download a report of displayed dashboard information.

Proposed solution

PAIMANA Prism provides a browser dashboard backed by a FastAPI service:

- React/Vite presents portfolio, queue, intelligence, simulator, and intake views.
- FastAPI protects project, scoring, and PAIMANA routes with an admin session.
- The backend retrieves and caches the public PAIMANA state summary.
- The risk module calculates a transparent baseline and can train Random Forest models when sufficient history is supplied.

- The recommendation layer matches a project profile against `data/teammate_interventions.json` using normalized names, categories, and token overlap.
- The frontend keeps teammate intervention-only records visibly separate from official PAIMANA project totals.

2. Complete System Workflow

The user journey implemented by the application is:

Login → Dashboard → Project data → Risk prediction → Explainable warning → AI recommendation → Decision support → Reports

Step 1: Login

When the React app loads, App calls `GET /api/auth/me`. While the result is pending it shows an access-checking state. If the session is not authenticated, it shows the login form. The form submits an admin phone number and password to `POST /api/auth/login`.

The backend compares the normalized phone number and a deterministic configured password format. On success it sets a signed `paimana_session` cookie. Protected routes require that cookie unless `AUTH_DISABLED=true` is configured.

Step 2: Dashboard

After authentication, `Dashboard` loads:

- `GET /api/paimana/states` for official state summary data.
- `GET /api/projects` for saved records and merged teammate intervention records.

The dashboard offers these views:

- **Portfolio overview:** official metrics, static demo sector chart, priority projects, and state analysis.
- **Early-warning queue:** project list ordered by the current client-side record order.
- **Project intelligence:** selected project trajectory, warning explanation, recommendation, and details.
- **What-if simulator:** assumed delay and cost-increase scenario.
- **Add project:** administrator intake and scoring flow.

Project data is refreshed from the project API every 15 seconds while the dashboard is open.

Step 3: Project data

The initial frontend contains four synthetic demonstration projects. Saved projects are loaded from the backend and merged with browser-local fallback records. The backend also reads the teammate intervention source of truth and adds unmatched entries as intervention-only records.

A project record contains fields such as ID, name, sector, state, cost, progress, status, and reported reason. The add/edit form also sends cost, schedule, milestone, and delay fields to the risk endpoint.

Step 4: Risk prediction

For a submitted record, `POST /api/risk/score` calls `score_record()` in `backend/risk_model.py`.

The result includes:

- `cost_risk`
- `time_risk`
- `priority`
- `method`
- plain-language `reasons`
- training readiness metadata
- a `recommendation` payload from the intervention engine

Risk scoring and recommendation generation are separate. The recommendation does not replace or alter the risk score.

Step 5: Explainable warning

The Project Intelligence page presents an **Explained Warning** section. It displays the current dashboard warning and supporting signals. In the current prototype, the headline and several values in this section are hard-coded demo presentation values rather than being dynamically generated from the selected project or API response.

The backend baseline does generate plain-language reasons from cost growth, time overrun, milestone achievement, and reported issue fields. Those backend reasons are returned by the scoring endpoint, but the current intelligence card does not fully replace its static explanatory copy with those response values.

Step 6: AI recommendation

For an official project, the intelligence view displays a recommendation card with:

- Predicted issue.
- Recommended action/solution.
- Matched category.
- Evidence or matched playbook/project.
- Confidence score.

The backend recommendation engine searches the teammate intervention JSON. The frontend also has a local keyword fallback for display when a project does not already carry a backend recommendation.

The current implementation is deterministic retrieval and keyword-rule matching. It is not an LLM, generative AI, embedding service, or vector database.

Step 7: Decision support

The user can open the simulator from Project Intelligence. The simulator exposes two controls:

- Additional delay in months.
- Assumed cost increase in percent.

It calculates scenario values in the browser. It does not call the backend risk model and it does not predict a guaranteed outcome.

Step 8: Reports

The dashboard has a PDF download action. `exportJudgeReport()` uses `jspdf` and `jspdf-autotable` in the browser to generate a report containing displayed national summary, project signals, and state-wise status information.

There are also functions for a text brief and an HTML official brief in `App.jsx`; the inspected UI does not expose visible call sites for those two functions. Reports are not server-side archived or signed.

3. Technical Architecture

Frontend

The frontend is a Vite React application in `frontend/`.

Important implementation areas:

- `frontend/src/main.jsx`: React entry point.
- `frontend/src/App.jsx`: authentication flow, dashboard state, API calls, components, project intake, recommendation display, simulator, and report functions.
- `frontend/src/index.css`: dashboard layout, panels, typography, responsive behavior, map, forms, and recommendation card styling.
- `frontend/package.json`: React, Recharts, jsPDF, jsPDF AutoTable, and India SVG map dependencies.

The app uses React state and effects. It does not use a frontend router; view selection is held in dashboard state.

Backend

The backend is a FastAPI application in `backend/main.py`.

Responsibilities include:

- Environment loading from `backend/.env` before authentication configuration is read.
- CORS configuration.
- Authentication and signed session cookies.
- Project storage and CRUD endpoints.
- Teammate intervention loading and merge behavior.
- Recommendation generation.
- Public PAIMANA state retrieval, caching, and stale fallback.
- Risk score and training endpoints.

`backend/risk_model.py` contains the scoring and training implementation.

APIs

Implemented routes:

Method	Route	Purpose
GET	/	Health-style root message.
POST	/api/auth/login	Validate admin credentials and set session cookie.

Method	Route	Purpose
GET	/api/auth/me	Return whether the current session is authenticated.
POST	/api/auth/logout	Delete the session cookie in the browser response.
GET	/api/projects	Return saved projects plus merged intervention records.
POST	/api/projects	Save or replace a project by ID.
PUT	/api/projects/{project_id}	Update an existing project without changing its ID.
POST	/api/risk/score	Score one record and attach a recommendation.
POST	/api/risk/train	Validate and train from a supplied history array.
GET	/api/paimana/states	Return cached or freshly fetched PAIMANA state data.
GET	/api/paimana/health	Report PAIMANA cache and last successful fetch status.

`api/index.py` imports `app` from `backend.main` for the deployment entry point. `vercel.json` provides the deployment rewrite configuration.

Data flow

1. The browser checks the session.
2. The browser requests official state data and project data.
3. The backend authenticates protected requests.
4. The project store returns saved records from a configured KV REST service or local JSON.
5. The backend merges `data/teammate_interventions.json` into the project response.
6. The add-project form posts a record to `/api/risk/score`.
7. The backend calls the risk model and recommendation engine.
8. The browser uses the score and recommendation response when saving/displaying the project.
9. The browser generates reports from data currently available in the dashboard.

Storage/database

The project store is implemented by `project_store_request()`:

- If `KV_REST_API_URL` and `KV_REST_API_TOKEN` are configured, the complete project list is stored under the Redis-compatible key `paimana:projects` through a REST request.
- Otherwise, the fallback is `data/processed/projects.json`.
- The frontend stores failed API additions/edits under the browser `localStorage` key `paimana-projects`.
- On reload, backend and local records are merged by project ID.

There is no relational database schema, migration system, database transaction layer, concurrency control, audit history, or server-side report archive.

AI/ML components

`risk_model.py` has two scoring modes:

1. **Transparent baseline:** formula-based cost and time risk from current record features.

2. **Random Forest comparison model:** two classifiers are trained when history meets the readiness conditions.

The recommendation engine is in `backend/main.py` as `generate_intervention_recommendation()`. It is a deterministic knowledge-base matcher, not a learned recommendation model.

4. Implemented Features

Authentication

What it does: Restricts project, risk, and PAIMANA data routes to an authenticated admin session.

Why required: Project editing and monitoring data should not be anonymously writable or readable in the current prototype.

How it works technically: Login normalizes the phone number, reads configured credentials from environment variables, and creates a base64-encoded payload signed with HMAC-SHA256. The signature and expiry are checked by `valid_session()`. The cookie is `HttpOnly` and `SameSite=Lax`. A configurable secure-cookie flag is available.

Important limitation: The password is deterministic from a configured prefix and phone suffix. There is no password hashing, rate limiting, account lockout, CSRF protection, role model, revocation list, or audit log. `AUTH_DISABLED=true` bypasses authentication and should not be used for a production deployment.

PAIMANA data integration

What it does: Fetches the official public PAIMANA state summary and shows national/state aggregates.

Why required: Decision-makers need a current official state-level context alongside project-level monitoring.

How it works technically: `fetch_paimana_state_data()` requests `https://paimana-proj.mospi.gov.in/Home/GetStateView`, searches the returned HTML for `var StateData = [...]`, and parses that JSON. `state_payload()` keeps an in-memory cache for the configured TTL, defaults to 30 minutes, and returns stale cached data if a later fetch fails.

Important limitation: Only the state summary endpoint is integrated. The project details used by the dashboard are not a full imported PAIMANA project archive. The repository documentation states that the available demo project data is synthetic and source PDFs are absent.

Dashboard

What it does: Provides one authenticated workspace for overview, queue, intelligence, simulator, and project intake.

Why required: Monitoring signals, explanations, interventions, and reporting need a single user journey.

How it works technically: `Dashboard` holds the selected view and project state, fetches API data with `useEffect`, refreshes projects every 15 seconds, and passes records to the view components. CSS provides the panel-based responsive layout.

Important limitation: Some overview values are static demonstration values, including sector chart data, queue count, report dates, and model-status text.

State-wise analysis

What it does: Shows official state totals, a map coloured by project count, hover details, and locally added project counts.

Why required: State and UT comparisons help identify geographic concentration and give context to portfolio monitoring.

How it works technically: `StateProjects` displays the official `states` and `national_total` payload. `StateMap` maps normalized state names to the `@svg-maps/india` locations, colors paths by `ProjectCount`, and updates the active state on hover, focus, or click.

Important limitation: The official totals are displayed from the PAIMANA response. Locally added project counts are overlaid in the frontend. Intervention-only records are excluded from the locally added official-project overlay.

Project intelligence page

What it does: Shows a selected project's identity, priority score, trajectory chart, warning explanation, recommendation, and latest details.

Why required: A ranked list alone does not tell a decision-maker what is happening or what action to consider.

How it works technically: `openProject()` stores the selected object and changes the view to `intelligence`. `Intelligence` renders a project hero, Recharts trajectory, Explained Warning section, recommendation card, detail strip, edit action, and simulator action.

Important limitation: The trajectory and several warning values are static demo content. The recommendation card is more dynamic than the warning copy, but it can use a frontend fallback when a backend payload is absent.

Risk prediction

What it does: Returns cost risk, time risk, combined priority, scoring method, reasons, and training metadata.

Why required: A consistent score helps rank projects for review and makes the dashboard actionable.

How it works technically: `score_record()` first calls `baseline_score()`. If `train_from_history()` is ready, it uses probability output from separate cost and time Random Forest classifiers. Otherwise it returns the transparent baseline. The priority is the average of cost and time risk.

Important limitation: The model is not a validated production predictor. The normal repository state may fall back to the baseline because the required sample CSV/data readiness is not reliably available. No trained model artifact is persisted.

Early-warning system

What it does: Provides an Early-warning queue and flags projects through risk pills and statuses such as Delayed, At risk, or Watching.

Why required: Reviewers need a fast way to identify records requiring attention.

How it works technically: The queue renders project records passed by **Dashboard**; risk pills use score thresholds, with critical at 75 or above and watch at 55 or above. The add-project flow maps score priority to At risk or Watching.

Important limitation: The queue controls labelled "All risks" and "All states" are visual controls only and do not currently filter the list. The queue count shown in navigation is static.

AI recommendation engine

What it does: Provides a predicted issue, recommended solution/action, category, matched project/playbook, and confidence value.

Why required: A risk signal is more useful when it is connected to a concrete mitigation option.

How it works technically: `generate_intervention_recommendation()` loads the JSON intervention list, tokenizes project name, sector, reason, category, and challenge text, and compares them with each intervention's name, category, problem, and solution. Exact normalized project-name matches receive extra weight, as do category, reason, and challenge overlaps. It returns the highest-scoring entry or a fixed generic fallback. `POST /api/risk/score` attaches this result without changing the risk score.

The frontend `getRecommendationForProject()` provides local keyword rules for common terms such as land acquisition, utility shifting, contractor capacity, right of way, environment, and payment when no recommendation object is present.

Important limitation: This is rule-based retrieval, not an LLM or generative AI system. Confidence is a heuristic value, not a calibrated probability. The current matching code does not use embeddings, a vector database, feedback learning, or external AI API.

Intervention-only projects

What it does: Displays teammate intervention records as supplementary records without pretending they are official PAIMANA observations.

Why required: The team's intervention list is useful as a practical playbook but must not inflate official project totals or receive invented official scores.

How it works technically: `merge_teammate_interventions()` reads `data/teammate_interventions.json`, normalizes project names, enriches matching existing projects, and appends unmatched entries with `sourceType: "team_intervention"` and `isInterventionOnly: true`. The frontend detects those flags, labels them "Intervention-only," shows N/A for risk/time, excludes them from official overlays, and renders their problem, solution, and category separately.

Important limitation: Intervention-only records have no official cost, time, progress, or PAIMANA state observation. Their zero-like display fields are not measured government values.

What-if simulator

What it does: Lets the user adjust assumed extra delay and assumed cost increase, then displays scenario cost/time risk and estimated impacts.

Why required: Decision-makers can discuss the possible cost of waiting and frame intervention conversations.

How it works technically: `Simulator` stores slider values in `Dashboard`. Scenario time is calculated as current time score plus `delay * 3`, capped at 99. Scenario cost is current risk plus `increase * 0.65`, capped at 99. Estimated cost impact is project cost multiplied by the assumed increase percentage.

Important limitation: The simulator is arithmetic and client-side. It does not call `/api/risk/score`, retrain a model, or produce a validated forecast.

PDF report generation

What it does: Downloads a client-generated PDF portfolio brief.

Why required: A reviewer or administrator may need a portable summary of the current dashboard view.

How it works technically: `exportJudgeReport()` creates a jsPDF document, writes national summary values, project signals, and state-wise data using `AutoTable`, adds page footers, and downloads the file with a date-based name.

Important limitation: The report reflects displayed dashboard data, includes static/demo values where the UI does, and is not an official government report. It is not server-archived, digitally signed, or independently generated from an audit snapshot.

5. AI/ML Explanation

Input data

The risk endpoint accepts a `record` object and an optional `history` array. The canonical history schema described by the repository includes observation month, project ID, cost fields, expenditure, completion dates, milestones, overruns, delay reason, status, and source report.

The current frontend submits a smaller record assembled from the intake form, including original cost, anticipated cost, delay reason, milestone values, progress, and dates. The recommendation engine additionally reads name, sector, category, challenge/problem, and reason text.

The repository currently includes `data/teammate_interventions.json` as the recommendation source. The documentation and code state that the original PAIMANA PDFs are not present and the demo project-month data is synthetic or unavailable in the current working state.

Risk score generation

`baseline_score()` derives features:

- Cost growth between original and anticipated cost.
- Expenditure ratio.
- Milestone achievement ratio.
- Time overrun months.
- Cost overrun percentage.
- A binary signal for a reported delay reason.

Cost risk uses cost growth, cost overrun, and delay-reason weights. Time risk uses time overrun, milestone gap, and delay-reason weights. Both are capped at 99. The model returns up to three plain-language reasons.

Model usage

`train_from_history()` creates forward-looking labels using the next available observation for each project:

- Cost target: next anticipated cost rises by at least 5%.
- Time target: next anticipated completion date is later.

It requires at least 20 valid observations and both target classes for both models. It then trains two `RandomForestClassifier` instances with `cost_overrun_percent` and `time_overrun_months` as features. `score_record()` uses positive-class probabilities when training is ready; otherwise it uses the transparent baseline.

No model object is persisted in `models/`. The training endpoint returns readiness metadata, not model artifacts.

Recommendation generation

Recommendation generation is knowledge-base matching:

1. Read intervention records from `data/teammate_interventions.json`.
2. Normalize project/profile text into tokens.
3. Score token overlap.
4. Add weight for exact normalized project-name matches.
5. Add weight for category, reason, and challenge/problem overlap.
6. Return the highest-scoring problem and solution with a heuristic confidence.
7. Use a fixed generic fallback when no intervention record exists.

The frontend has a separate keyword fallback for common risk reasons. Both paths preserve the existing risk score.

Limitations

- Recommendations are not generated by an LLM or embedding model.
- Recommendation confidence is not statistically calibrated.
- The intervention JSON is a finite knowledge base and may not cover new project types.
- Random Forest training is conditional and uses only two model features.
- There are no reported model evaluation metrics in the running product.
- There is no cross-validation, temporal holdout evaluation, calibration, SHAP display, or feature-importance UI.
- The dashboard contains synthetic/static demo values in several areas.
- The current PAIMANA integration is a state-summary fetch, not a full archival project ingestion pipeline.
- The what-if simulator is a simple arithmetic scenario tool.
- Missing, inconsistent, or poorly formatted project fields can reduce score and matching quality.

Future improvements

These are future improvements, not current capabilities:

- Replace synthetic observations with validated, source-linked PAIMANA extracts.
- Build the documented PDF/table extraction and validation pipeline.
- Preserve report and page provenance for every observation.
- Add temporal train/test evaluation and publish precision, recall, F1, ROC-AUC, calibration, and drift metrics.
- Expand model features and compare transparent baseline, Random Forest, and other validated models.
- Add embeddings/vector retrieval for intervention matching and evaluate it against keyword matching.
- Add human feedback, recommendation acceptance/outcome tracking, and versioned intervention playbooks.
- Replace static intelligence values with live API-derived evidence.
- Make queue filters and prioritization dynamic.
- Persist model artifacts and recommendation snapshots with audit history.
- Add stronger authentication, authorization, rate limiting, CSRF protection, secret rotation, and audit logging.

6. Live Demo Script: 5-7 Minutes

Opening: 30-45 seconds

"Good morning. Our project is PAIMANA Prism, a decision-support dashboard for infrastructure project monitoring. The problem is not only that project information exists in reports; the problem is turning it into a clear next decision. Our prototype connects monitoring data, risk signals, explanations, intervention knowledge, and a what-if view in one workflow."

Problem explanation: 45-60 seconds

"Project reviewers need to answer three questions quickly: Which project needs attention, why is it at risk, and what intervention should be considered? Periodic reports and state summaries provide evidence, but reviewing them manually makes comparison and action difficult. PAIMANA Prism is designed to put those signals into a structured review flow. We also clearly distinguish official PAIMANA state data from synthetic demonstration project data and teammate intervention records."

Login and dashboard: 45-60 seconds

"First, I sign in with the configured admin credentials. The application checks the session through the FastAPI backend. After login, the overview shows the official PAIMANA state summary when the public portal is available. The dashboard also shows portfolio signals, a priority list, and the state-wise analysis area. The official totals are kept separate from supplementary intervention-only records."

Project intelligence: 90 seconds

"I will open a project from the priority area. Project Intelligence shows the selected project, its current priority score, a cost and expenditure trajectory, and the Explained Warning section. The warning section provides a readable explanation of the signals shown in the prototype. Below that is the AI intervention recommendation card. It separates the predicted issue, recommended action, matched category, evidence or matched playbook, and confidence score."

Technically, the score and recommendation are separate. The risk endpoint calls the existing scoring module, then attaches a recommendation produced by matching the project profile against our teammate intervention

knowledge base. This recommendation layer does not change the project risk score.”

Recommendation honesty: 30 seconds

“When presenting this feature, we describe it accurately as deterministic, rule-based recommendation retrieval. It is an AI-style decision-support layer in the product experience, but the current implementation is not an LLM or generative model. That distinction matters because the recommendation is explainable and auditable, but its confidence is heuristic.”

Add project and simulator: 75-90 seconds

“Next, I open Add Project. The form captures identity, costs, schedule, progress, milestones, and delay reason. When submitted, the frontend sends the record to `/api/risk/score`. The backend returns cost risk, time risk, priority, reasons, and recommendation data. The project can then be saved through the project API.

From Project Intelligence I can open the what-if simulator. I adjust assumed additional delay and assumed cost increase. The browser calculates the scenario response and estimated impacts. This is decision support for discussion, not a guaranteed forecast and not a second model training path.”

Reports and technical explanation: 60-75 seconds

“Finally, I download the portfolio brief. The browser generates a PDF with the displayed national summary, project signals, and state-wise status. The backend uses FastAPI, signed session cookies, JSON/KV project storage, PAIMANA state retrieval with in-memory caching, and the risk module. The model has a transparent baseline and a conditional Random Forest mode when the required labelled history is available.”

Closing: 30-45 seconds

“Our key contribution is the connected review workflow: data context, risk signal, explanation, intervention recommendation, scenario discussion, and reporting. We have intentionally preserved data provenance boundaries: official PAIMANA aggregates are not mixed with synthetic demo records, and teammate intervention records are labelled supplementary. The next step is validating the full PAIMANA archive, evaluating the models on later reporting periods, and adding audited human feedback to improve recommendations.”

7. Jury Questions and Answers

AI and ML

1. Is this using generative AI or an LLM?

Not in the current implementation. The recommendation engine is deterministic matching against `teammate_interventions.json`, with token overlap, normalized project-name matching, category matching, and a rule-based fallback. The risk module can use Random Forest classifiers when training conditions are met.

2. Why call it an AI recommendation engine?

It is an AI-style decision-support layer because it maps project evidence to intervention guidance. For technical accuracy, we should say the current version is a rule-based and knowledge-base recommendation

engine, not a generative AI system.

3. How is the risk score generated?

The baseline uses cost growth, expenditure ratio, milestone achievement, time overrun, cost overrun, and delay-reason signals. When enough labelled history exists, two Random Forest classifiers estimate cost and time deterioration probabilities.

4. What are the model inputs?

The training model currently uses `cost_overrun_percent` and `time_overrun_months`. The transparent baseline uses additional cost, expenditure, milestone, and delay-reason fields. Recommendation matching uses project name, sector/category, reason, challenge/problem, and intervention text.

5. What is the prediction target?

Cost deterioration is a next-observation anticipated-cost increase of at least 5%. Time deterioration is a later anticipated completion date in the next available observation for the same project.

6. Why use Random Forest?

It is a practical comparison model for tabular data, can model non-linear relationships, and works with a small feature set. The prototype also retains a transparent baseline so scoring remains explainable when training data is insufficient.

7. How do you prevent future-data leakage?

Forward values are used to construct labels, while current-month values are used as features for scoring. The repository's planned validation guidance calls for time-based train/test splitting. A complete production evaluation is not yet implemented in the dashboard.

8. How is recommendation confidence calculated?

The backend derives a heuristic confidence from the match score, with boosts for exact project-name, category, reason, and challenge overlap. It is not a calibrated probability and should not be presented as statistically validated confidence.

9. What happens when there is no matching intervention?

The backend returns a fixed generic recommendation. The frontend also has local keyword rules for common reasons such as land acquisition, utility shifting, contractor capacity, right of way, environment, and payment.

10. How would you improve the recommendation engine?

Validate the current matcher against human-reviewed outcomes, add embeddings/vector retrieval, record accepted or rejected recommendations, version the playbook, and evaluate recommendation precision and usefulness over time.

Data

11. Is all data in the dashboard official PAIMANA data?

No. The state summary is fetched from the public PAIMANA portal when available. The repository describes the project demonstration records as synthetic, and teammate intervention entries are supplementary intervention-only records.

12. Where is the PAIMANA data fetched from?

The backend requests the public [Home/GetStateView](#) page and extracts its [StateData](#) JavaScript array. It caches the parsed response in process memory.

13. Do you have the full PAIMANA PDF archive?

No. The repository documents that source PDFs are not present. The extraction workflow is documented as a plan, not as a completed ingestion pipeline.

14. How do you preserve provenance?

The intended canonical schema includes source report fields, and the PAIMANA API response records its source URL and retrieval time. The current product does not yet provide complete per-row PDF page provenance because the full extraction archive is not integrated.

15. Why keep intervention-only records separate?

They are teammate recommendations, not official observations. Treating them as monitored PAIMANA projects would create invented totals, costs, progress, or risk values. The frontend labels them and excludes them from official overlays.

Technical

16. What is the technology stack?

React 18 with Vite, Recharts, jsPDF, jsPDF AutoTable, and an India SVG map on the frontend; FastAPI, Pydantic, pandas, scikit-learn, and python-dotenv on the backend.

17. How does the frontend communicate with the backend?

It uses browser [fetch\(\)](#) calls to the authentication, project, risk, and PAIMANA endpoints. Credentials are included so the session cookie is sent.

18. Where are projects stored?

The backend uses a configured Redis-compatible REST KV store when credentials exist, otherwise [data/processed/projects.json](#). The frontend uses localStorage as a fallback for failed project saves.

19. Does the simulator use the ML model?

No. It uses simple browser arithmetic based on the selected project's current values and slider assumptions. This is explicitly scenario framing, not a validated model forecast.

20. What happens if PAIMANA is unavailable?

The backend returns a stale cached response if one exists. If there is no cache, the API returns an error. The frontend shows an unavailable/stale state message.

21. How are project names matched?

The backend applies Unicode normalization, lowercases text, replaces punctuation and dashes, removes non-alphanumeric characters, and compares normalized names. Recommendation matching also uses token overlap.

22. What are the current API protections?

Protected routes call `require_session()`. Sessions are signed with HMAC-SHA256 and expire. Cookies are HttpOnly and SameSite=Lax. This is prototype security; it does not yet include rate limiting, CSRF protection, role-based authorization, password hashing, or token revocation.

Security

23. Where are credentials configured?

Environment variables loaded from `backend/.env` configure admin accounts, session secret, cookie behavior, CORS, cache TTL, and optional KV storage. Secrets must not be committed or exposed.

24. What security improvements are needed before production?

Use a proper identity provider or hashed password store, rotate secrets, remove sensitive `.env` material from version control, add rate limiting and lockout, add CSRF protection, use HTTPS secure cookies, add roles and audit logs, validate input more strictly, and revoke sessions when needed.

25. What does logout do?

It asks the backend to delete the browser cookie. Because the current session token is stateless, logout does not revoke a copied token before its expiry.

Scalability

26. Can this handle the complete national PAIMANA archive?

The current API shape can be extended, but the repository does not yet contain the full archive, extraction pipeline, persistent model artifacts, or production database design needed to claim national-scale readiness.

27. What is the main storage scalability issue?

The current store reads and writes the entire project list under one KV key or one JSON file. That is simple for a prototype but not suitable for high-concurrency updates or large historical observations.

28. How would you scale it?

Use a database with project and observation tables, indexed project IDs and months, object storage for raw reports, a background extraction pipeline, persistent cache, job queue for training, model registry, API pagination, and horizontally scalable stateless services.

Innovation and impact

29. What is innovative about the project?

The product connects multiple decision steps in one interface: official state context, risk scoring, explanation, intervention playbook matching, scenario discussion, and reporting. It also demonstrates responsible data separation between official aggregates, synthetic demo records, and teammate interventions.

30. What impact can you claim today?

We can claim a working prototype and demonstrated workflow. We should not claim improved government outcomes, validated prediction accuracy, complete PAIMANA coverage, calibrated recommendation confidence, or production-scale deployment until those are measured and implemented.

8. Team Member Preparation

What every presenter should know

Every presenter should be able to explain:

- PAIMANA Prism is a working prototype, not a complete official data archive.
- The frontend is React/Vite and the backend is FastAPI.
- The state summary is fetched from the public PAIMANA portal and cached in memory.
- The project store uses KV REST when configured and JSON fallback otherwise.
- The risk endpoint returns cost risk, time risk, priority, method, reasons, and recommendation data.
- Random Forest activates only with enough valid labelled history and both target classes; otherwise the transparent baseline is used.
- Recommendation matching is deterministic retrieval from the teammate intervention JSON.
- Intervention-only records are supplementary and excluded from official totals and risk display.
- The simulator uses client-side arithmetic and is not a model forecast.
- The PDF is generated in the browser from displayed data.
- Security is suitable for demonstrating the flow but requires hardening before production.

Suggested responsibility split

Product/problem presenter: Explain the monitoring problem, user journey, decision-maker value, and data-boundary discipline.

Frontend presenter: Demonstrate login, overview, state map, project intelligence, recommendation card, simulator, and PDF action. Explain that some visual values are demo/static values.

Backend presenter: Explain routes, session cookie, project CRUD, PAIMANA fetch/cache, fallback storage, and recommendation attachment.

ML/data presenter: Explain baseline features, forward labels, Random Forest readiness, recommendation token matching, synthetic-data limitation, and validation roadmap.

Security/deployment presenter: Explain environment configuration, signed cookies, CORS, Vercel entry point, local setup, and the security gaps that must be closed before production.

Claims that should not be made

Do not say:

- "The system uses a trained deep-learning or generative AI model."

- "The recommendation confidence is a calibrated probability."
- "All displayed project records are official PAIMANA records."
- "The full PAIMANA PDF archive has been ingested."
- "The model has proven accuracy" without evaluation results.
- "The simulator predicts the actual future cost or completion date."
- "The dashboard automatically prioritizes projects using a validated national ranking."
- "The PDF is an official government report."
- "The system is production-secure."
- "The displayed 87%, 14.6%, 2x, queue count, or sector chart values are all live model outputs."

What to say about limitations

Use this answer:

"This is a functional prototype. The public PAIMANA state summary integration is real, while the project demonstration records are clearly labelled synthetic and the intervention list is supplementary. Our current risk layer has a transparent baseline and a conditional Random Forest path, but it needs validated historical PAIMANA observations and temporal evaluation before production claims. The recommendation engine is explainable rule-based retrieval today; our next step is evaluating it with human feedback and adding stronger retrieval and audit capabilities."

Final Codebase Review Checklist

Before presenting, verify:

- Backend starts with the intended `backend/.env` values and secrets are not exposed.
- The frontend can reach the FastAPI base/API proxy.
- Login works or the demo environment explicitly uses the configured authentication mode.
- `GET /api/paimana/states` either loads official state data or displays the stale/unavailable message honestly.
- Project Intelligence shows the recommendation card below Explained Warning.
- Intervention-only records show N/A risk/time and the non-PAIMANA label.
- A project submitted through Add Project receives a score response and saves correctly.
- The simulator is described as a scenario calculator.
- The PDF contains the currently displayed values.
- No presenter describes synthetic records, static warning values, or deterministic rules as validated official AI results.